

How the deep learning model is created for given parameters:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, SimpleRNN, Dense, Dropout
```

- Import models and layers from tensorflow keras

```
def create_model(num_layers = 4, units_per_layer = 256, layer_name = LSTM ,
num_time_steps = 20, number_of_features=5, activation="ReLU",
loss="mean_absolute_error", optimizer="rmsprop",
metrics="mean_absolute_error"):
```

- Define a function create_model that accepts various parameters like number of layers, neurons per layer, num of time steps to look at, num of features, type of layer, loss, optimisers and metrics.

```
dl_model = Sequential()
```

- Define a Sequential model

```
if (layer == 0):
    # input_shape=(batch_size, num_time_steps/data_seq_length/rows,
    number_of_features/columns)
    '''
    Sequence Length = 5:
        You choose a sequence length of 5, which means the model will look at 5
consecutive days of stock data at a time to make a prediction.
        For example, for a given training instance, if your data looks like this:
        Day    Open    High    Low    Close    Volume
        Day 1   100     105    98 102    200000
        Day 2   102     106    101   104    180000
        Day 3   104     107    102   106    170000
        Day 4   106     109    103   108    150000
        Day 5   108     110    104   109    140000

    Batch Size = 32:
        If your batch size is 32, it means the model processes 32 sequences at a
time before it updates its weights.
        So, for a batch of size 32, the model will take 32 such sequences, each
consisting of 5 consecutive days of data, and process them together.
        For instance, the first batch might contain these 32 sequences:

    Batch 1      Sequence Length (5 Days Each)
    Sequence 1 Day 1 to Day 5
    Sequence 2 Day 2 to Day 6
    Sequence 3 Day 3 to Day 7
```

```

Sequence 4 Day 4 to Day 8
...
Sequence 32 Day 32 to Day 36

How They Work Together:
    Sequence Length (5): Determines how much "history" the model looks at to
make a prediction. In this case, 5 days of stock data.
    Batch Size (32): Determines how many sequences are processed together
before the model updates its weights. In this case, 32 sequences, each 5 days
long, are processed in one batch.
    Currently, batch size set to none, so that we can dynamically change the
batch size.
'''
    dl_model.add(layer_name(units_per_layer, batch_input_shape=(None,
num_time_steps, number_of_features), return_sequences=True))

elif (layer == num_layers - 1):
    # last layer, return sequences is false cuz we not stacking anymore.
    dl_model.add(layer_name(units_per_layer, return_sequences=False))
else:
    # hidden layers return sequences so that we can stack lstm layers on top of
each other
    dl_model.add(layer_name(units_per_layer, return_sequences=True))

```

- Use if conditional, decide whether to return sequences from the stacked layer or not.
- For example, if it's the first layer of the stack, then set batch_input_shape as well, basically we tell the model how our data is used for training, like batch size, sequence length and number of features.
- If it's the last layer, we ask the model to not return sequence, because we are not stacking after this layer.
- For other layers, they are hidden layers, so we ask the model to return sequences and use that for further stacking. But in this case we don't specify the batch_input_shape again. We only need to do that for the first layer.

```
dl_model.add(Dropout(0.2))
```

- Add a dropout layer that deactivates a specified percentage of randomly chosen neurons. This is useful in preventing overfitting.

```
dl_model.add(Dense(1, activation=activation))
```

- This is the output layer, Dense means it is fully connected and 1 means it has 1 neuron with the specified activation function.

```
dl_model.compile(loss=loss, optimizer=optimizer, metrics=[metrics])
```

compile the model with given loss function, optimizer and metrics. loss is the fn to be minimised, i.e the goal for dl_model is to minimise the loss.

```
optimizer is the algorithm to be used to minimise the loss. metrics is the fxn to be used to evaluate/track the model; often same as loss. .
```

- This line compiles the model with the given loss, optimiser and metric.
- After successful compilation, our model is ready for training with the given parameters.

Use the above function to experiment with different DL networks (e.g., LSTM, RNN, GRU, etc.) and with different hyperparameter configurations (e.g. different numbers of layers and layer sizes, number of epochs, batch sizes, etc.)

I wrote a function that calls the `create_model(...)` function and compiles different models based on randomly selected parameters selected from a specified domain.

```
def rand_parameters():
    num_layers = random.randint(4, 10)
    print(num_layers)

    units_per_layer = random.randint(128, 256)
    print(units_per_layer)

    layer_name = random.choice([LSTM, GRU, SimpleRNN])
    print(layer_name)

    num_time_steps = random.randint(10, 50)
    print(num_time_steps)

    number_of_features = random.randint(5, 10)
    print(number_of_features)

    activation = random.choice(["ReLU", "tanh", "sigmoid"])
    print(activation)

    loss = random.choice(["mean_absolute_error", "mean_squared_error",
"mean_squared_logarithmic_error"])
    print(loss)

    optimizer = random.choice(["Adam", "RMSprop", "SGD"])
    print(optimizer)
```

```

    metrics = random.choice(["mean_absolute_error", "mean_squared_error",
"mean_squared_logarithmic_error"])
    print(metrics)
    return [num_layers, units_per_layer, layer_name, num_time_steps,
number_of_features, activation, loss, optimizer, metrics]

```

- This function generates random values for different parameters like num_layers, units_per_layers, layer_name, num_time_steps etc and then return those values.

```

def build_toy_models(num_of_models):
    list_of_models = []
    for model in range (num_of_models):

        [num_layers, units_per_layer, layer_name, num_time_steps,
number_of_features, activation, loss, optimizer, metrics] = rand_parameters()

        dl_model = create_model(num_layers, units_per_layer, layer_name,
num_time_steps, number_of_features, activation, loss, optimizer, metrics)
        list_of_models.append(dl_model)
    return list_of_models

num_of_models = random.randint(3, 10)
print(num_of_models)
model_list = build_toy_models(num_of_models)

```

- This function calls the rand_parameters function a specified number of times based on the value of num_of_models and then generates many models, again based on the value of num_of_models. For example if num_of_models is 5, then this function will create 5 models with randomly chosen parameters as explain for the previous function.
- Then the function appends these different models to a list called list_of_models.
- To note: keras names layers globally to ensure uniqueness. In the below example, we have 3 models, 2 Simple RNNs and 1 GRU. The first Simple RNN has 7 layers. And the second simple RNN also has 7 layers. But notice that second RNN names start from 7, not 1. This is because of global naming by keras. To disable global naming, we can use the following code:

```

# from tensorflow.keras import backend as k
# k.clear_session() # Clear the session to reset layer naming, this will
be put in the for loop so that naming order resets at each iteration.

```

```
[model.summary() for model in model_list]
```

- This line prints a summary of the models.

The summary of three models is:

```
Model: "sequential"
```

```
Layer(type)
Output
Shape
Param #
== == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == =
simple_rnn(SimpleRNN)(None, 43, 136)
19448

simple_rnn_1(SimpleRNN)(None, 43, 136)
37128

simple_rnn_2(SimpleRNN)(None, 43, 136)
37128

simple_rnn_3(SimpleRNN)(None, 43, 136)
37128

simple_rnn_4(SimpleRNN)(None, 43, 136)
37128

simple_rnn_5(SimpleRNN)(None, 43, 136)
37128

simple_rnn_6(SimpleRNN)(None, 136)
37128

dropout(Dropout)(None, 136)
0

dense(Dense)(None, 1)
137

== == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == =
Total
params: 242353(946.69
KB)
Trainable
params: 242353(946.69
KB)
Non - trainable
params: 0(0.00
Byte)
```

```
Model: "sequential_1"
```

```
Layer(type)
```

[illegible]

```
Param #
== == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == =
gru (GRU) (None, 19, 249)
191979

gru_1 (GRU) (None, 19, 249)
373500

gru_2 (GRU) (None, 19, 249)
373500

gru_3 (GRU) (None, 19, 249)
373500

gru_4 (GRU) (None, 19, 249)
373500

gru_5 (GRU) (None, 19, 249)
373500

gru_6 (GRU) (None, 19, 249)
373500

gru_7 (GRU) (None, 249)
373500

dropout_2 (Dropout) (None, 249)
0

dense_2 (Dense) (None, 1)
250
```